

ADR-002-distributed-worker-architecture

ADR-002: Distributed Worker Architecture

Status

Accepted

Date

2026-01-08

Context

HelixCode requires a distributed computing architecture to handle resource-intensive AI development tasks. The platform needs to:

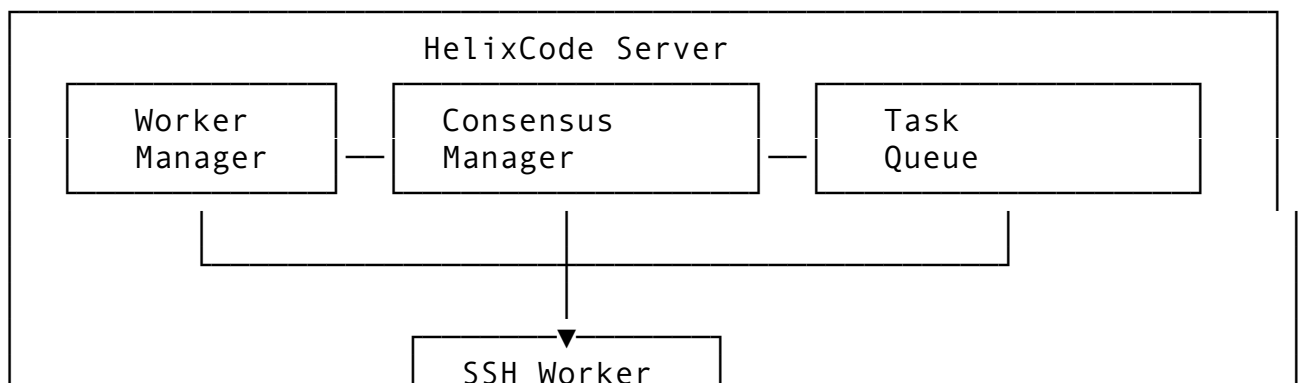
1. **Scale horizontally:** Support multiple worker nodes for parallel task execution
2. **Support heterogeneous hardware:** Workers may have different capabilities (GPU, high memory, specialized tools)
3. **Handle network failures gracefully:** Tasks should survive worker disconnections
4. **Auto-provision workers:** New workers should be automatically configured when added
5. **Ensure security:** SSH-based connections must be secure and verified
6. **Provide isolation:** Tasks from different users/projects should be isolated
7. **Support capability-based routing:** Tasks should be assigned to workers with appropriate capabilities

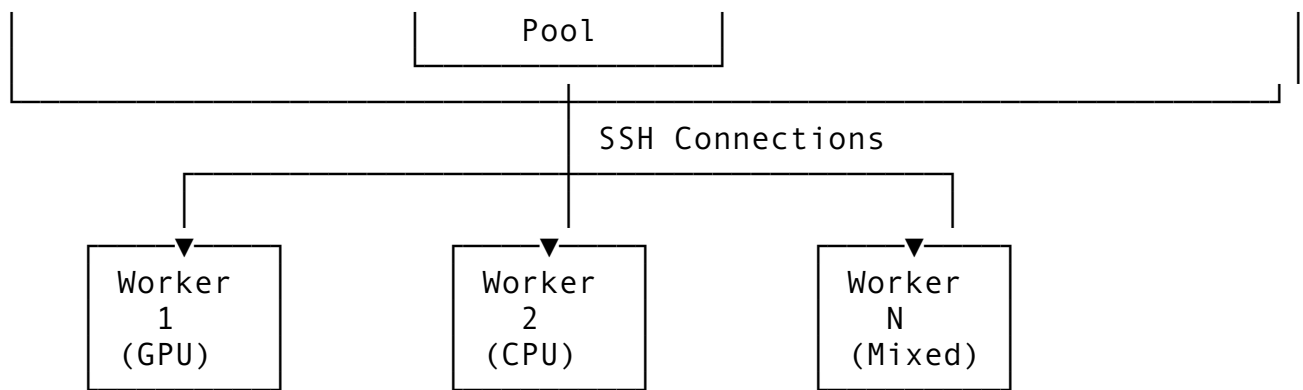
The challenge was designing a system that provides enterprise-grade reliability while remaining simple to operate and extend.

Decision

We implemented an SSH-based distributed worker pool architecture with the following components:

Core Architecture





SSH Worker Pool

The SSHWorkerPool manages connections to distributed workers:

```

type SSHWorkerPool struct {
    workers      map[uuid.UUID]*SSHWorker
    autoInstall  bool
    cliDownloadURL string
    hostKeys     *HostKeyManager
    isolation     *WorkerIsolationManager
    consensus    *ConsensusManager
}
  
```

Key features: - **Secure SSH connections** with host key verification - **Auto-installation** of Helix CLI on new workers - **Capability detection** (CPU, memory, GPU, Docker, CUDA) - **Health monitoring** with configurable intervals (default 30s) - **Sandbox isolation** for task execution

Worker Configuration

Workers are configured via the pool configuration:

```

type WorkerConfigEntry struct {
    Host      string `json:"host"`
    Port      int    `json:"port"`
    Username  string `json:"username"`
    KeyPath   string `json:"key_path"`
    Capabilities []string `json:"capabilities"`
    DisplayName string `json:"display_name"`
}
  
```

Security Model

1. **Host Key Verification:** Prevents man-in-the-middle attacks
2. **SSH Key Authentication:** Preferred over passwords
3. **Sandboxed Execution:** Tasks run in isolated environments
4. **Resource Limits:** Memory and CPU limits per task
5. **Secure Ciphers:** AES-256-CTR and strong MACs only

```
sshConfig := &ssh.ClientConfig{
    HostKeyCallback: p.hostKeys.VerifyHostKey(),
    Config: ssh.Config{
        Ciphers: []string{"aes128-ctr", "aes192-ctr", "aes256-ctr"},
        MACs:    []string{"hmac-sha2-256-etm@openssh.com", "hmac-sha2-256"},
    },
}
```

Task Distribution

Tasks are distributed based on: 1. **Criticality level:** low, normal, high, critical 2. **Priority:** Numeric priority for ordering 3. **Capabilities:** Required worker capabilities 4. **Resource requirements:** CPU, memory, GPU needs 5. **Current worker load:** Task count vs capacity

Health and Recovery

- **Heartbeat-based health:** Workers must report within TTL (default 30s)
- **Automatic status transitions:** healthy -> degraded -> unhealthy -> offline
- **Task checkpointing:** Preserves work across worker failures
- **Task reassignment:** Failed tasks can be reassigned to healthy workers

Consensus Management

For multi-master deployments, a consensus manager handles: - Leader election - State synchronization - Split-brain prevention

Consequences

Positive

1. **Scalability:** Can scale from single node to hundreds of workers
2. **Flexibility:** Workers can have varied capabilities and hardware
3. **Resilience:** Tasks survive worker failures through checkpointing
4. **Security:** Strong SSH security with host key verification
5. **Operability:** Auto-installation reduces setup friction
6. **Isolation:** Sandboxing prevents task interference
7. **Cost Efficiency:** Can use spot instances or heterogeneous hardware

Negative

1. **Network Dependency:** Requires reliable network connectivity
2. **SSH Overhead:** SSH connection establishment has latency
3. **Configuration Complexity:** Multiple workers require careful configuration
4. **Monitoring Overhead:** Health checks consume resources
5. **Security Surface:** SSH access requires careful key management

Neutral

1. **Learning Curve:** Operations team needs SSH and security expertise

2. Infrastructure Cost: Each worker node has infrastructure costs

Alternatives Considered

Alternative 1: Kubernetes-Based Worker Pods

Description: Use Kubernetes to manage worker containers.

Pros: - Native scaling and scheduling - Built-in health monitoring - Container isolation - Easy deployment

Cons: - Kubernetes infrastructure requirement - Higher resource overhead - Complex for GPU workloads - Less control over node placement

Why Rejected: Many enterprises have existing bare-metal or VM infrastructure. SSH-based approach works with any Linux machine without requiring Kubernetes.

Alternative 2: gRPC-Based Communication

Description: Use gRPC instead of SSH for worker communication.

Pros: - Higher performance - Bidirectional streaming - Strong typing with protobuf - Built-in load balancing

Cons: - Requires custom daemon on workers - Less secure by default - Additional protocol to manage - Firewall configuration needed

Why Rejected: SSH provides universal access to any Linux machine and includes security by default. No additional daemon installation required.

Alternative 3: Agent-Based Architecture (Like Ansible)

Description: Deploy agents on workers that poll for tasks.

Pros: - Works through firewalls (outbound only) - Simpler network configuration - No SSH key management

Cons: - Polling latency for task assignment - Agent deployment and updates - Less real-time control - Additional component to maintain

Why Rejected: Real-time task execution and monitoring require bidirectional communication. SSH provides this without additional agents.

Alternative 4: Message Queue Distribution (RabbitMQ/Kafka)

Description: Use message queues for task distribution.

Pros: - Decoupled architecture - Built-in persistence - Easy horizontal scaling - Natural load balancing

Cons: - Additional infrastructure - Message queue management - Higher latency for interactive tasks - Complex error handling

Why Rejected: Interactive development workflows require low-latency direct communication. Message queues add unnecessary complexity and latency.

Implementation Notes

- Worker implementations are in `internal/worker/`
- SSH pool manages connection lifecycle
- Health checks run in background goroutines (30s default interval)
- Sandbox cleanup runs hourly to remove expired sandboxes
- CLI download URL is configurable via `HELIX_CLI_DOWNLOAD_URL` env var

Related Decisions

- ADR-001: LLM Provider Interface (tasks may use LLM providers)
- ADR-004: Workflow Execution Model (workflow steps execute on workers)
- ADR-006: Database Schema Design (worker state persisted to database)

References

- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/worker/ssh_pool.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/worker/manager.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/worker/types.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/worker/isolation.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/worker/consensus.go`